

Faculty of Engineering
Department of Electrical Engineering

DSP Project report
ECG filtering & Analysis

Prepared by:

Nouran Ehab Henedak Mohamed
Menna Allah Bahaa Abuzaid

Supervisor:

DR/ Doaa Gamal

Introduction:

Electrocardiogram (ECG) signals represent the electrical activity of the heart and contain essential diagnostic features such as the P wave, QRS complex, and T wave. ECG signals play a vital role in the diagnosis and monitoring of cardiovascular diseases; however, the accuracy and reliability of clinical analysis are often affected by various types of noise and interference.

Common noise sources that degrade ECG signal quality include baseline drift caused by patient movement or respiration, and muscle (EMG) artifacts. These noise components can significantly distort the ECG waveform, making it difficult to accurately extract clinically relevant information. Consequently, effective signal processing techniques are required to enhance ECG signal quality while preserving its morphological characteristics.

This project demonstrates the practical application of Digital Signal Processing (DSP) concepts to ECG signal enhancement. The implemented methods include time-domain analysis, frequency-domain analysis using the Fast Fourier Transform (FFT), digital filter design, magnitude and phase response evaluation, and group delay analysis. Through these techniques, the project highlights the effectiveness of DSP-based filtering approaches in reducing noise and improving the interpretability of ECG signals.

Project Overview:

```
# (0) IMPORT LIBRARIES
# =====
import numpy as np
import matplotlib.pyplot as plt
import scipy.signal as signal
from scipy.signal import butter, firwin, freqz, filtfilt, sosfiltfilt, sosfreqz
from scipy.io.wavfile import write as wav_write
import wfdb # Library for reading PhysioNet data
```

1- Loading real ECG signal:

A real ECG signal from the **MIT-BIH Arrhythmia Database** was used in this project. The signal was sampled at 360 Hz and contains clear cardiac features (P, QRS, and T waves) & common types of noise:

- **Baseline Wander:** Low-frequency interference resulting primarily from respiratory motion or poor electrode contact.
- **Muscle Tremor (Electromyography - EMG):** High-frequency noise originating from muscle activity.

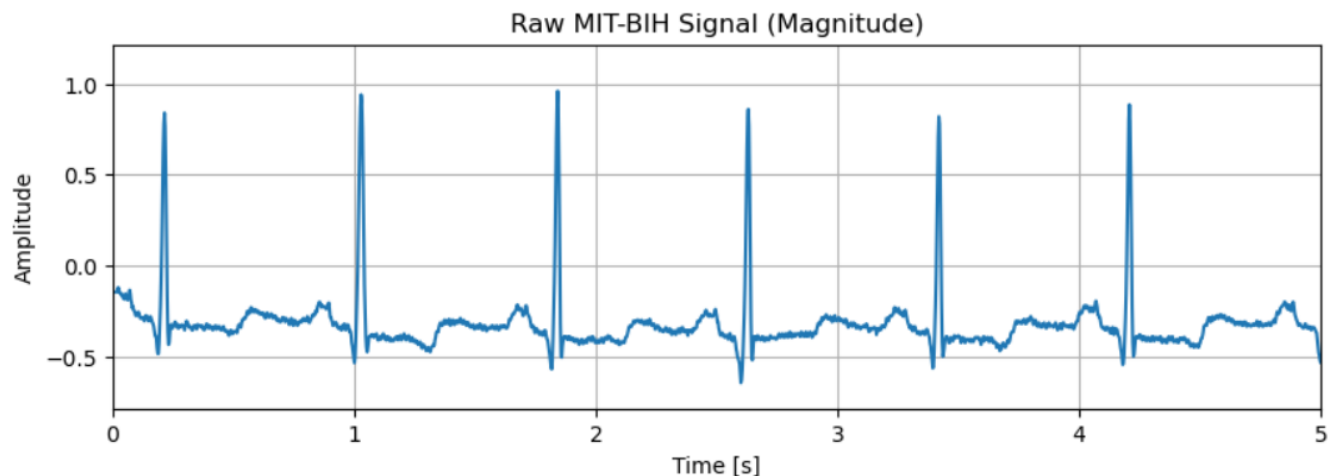
The raw ECG signal was considered as the input signal for analysis and filtering.

```
record_name = '100'
# Load the record. We take 100 seconds (360 samples/sec * 100 sec = 36000 samples)
record = wfdb.rdrecord(record_name, samp_to=36000, pn_dir='mitdb/')

# Extract the sampling frequency
fs = record.fs          # fs = 360 Hz
duration = len(record.p_signal) / fs
t = np.arange(0, duration, 1/fs)
N = len(t)
# Extract Lead II signal (Column 0) as the clean reference
ecg_noisy = record.p_signal[:, 0]
print(f"Loaded MIT-BIH Record: {record_name}. Fs: {fs} Hz. Length: {duration:.2f} s")

plt.figure(figsize=(10,3))
plt.plot(t, ecg_noisy)
plt.title("Raw MIT-BIH Signal (Magnitude)")
plt.xlabel("Time [s]")
plt.ylabel("Amplitude")
plt.xlim(0, 5)
plt.grid(True)
plt.show()
```

Loaded MIT-BIH Record: 100. Fs: 360 Hz. Length: 100.00 s



2- FFT Analyses:

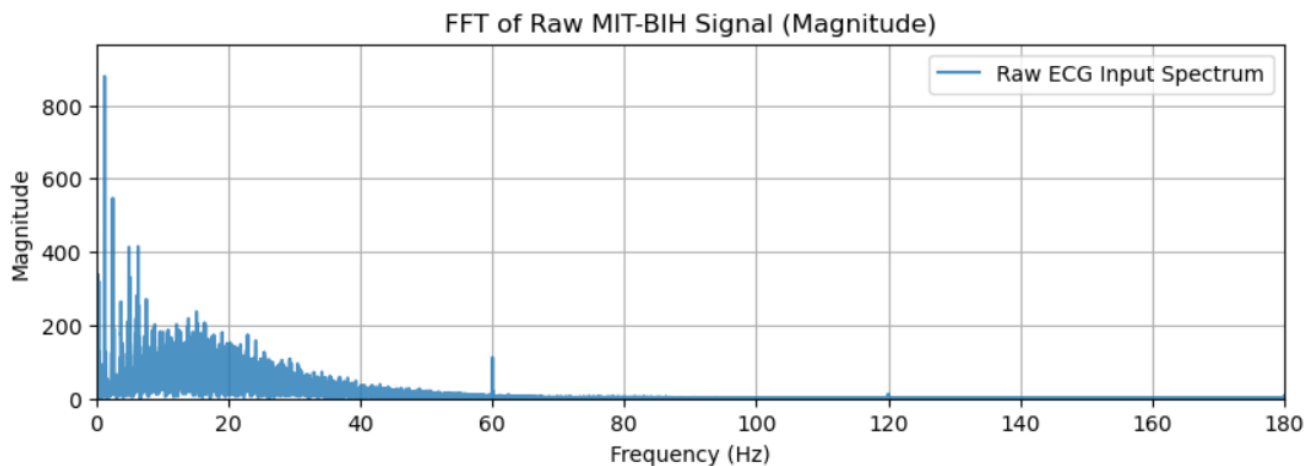
```
# Set the signal to be analyzed (the raw input signal from MIT-BIH)
signal_to_analyze = ecg_noisy
N = len(signal_to_analyze)

# Calculate the FFT (using rfft for real signal efficiency)
# fft_signal contains the magnitude
fft_signal = np.abs(np.fft.rfft(signal_to_analyze))
# Calculate the corresponding frequencies
freq = np.fft.rfftfreq(N, 1/fs)

# Drawing FFT plot
plt.figure(figsize=(10, 3))

# Plotting the positive half of the spectrum
plt.plot(freq, fft_signal, label="Raw ECG Input Spectrum", alpha=0.8)

# Setting limits and labels
plt.xlim(0, fs/2) # Display up to the Nyquist frequency
plt.ylim(0, np.max(fft_signal[1:]) * 1.1) # Adjusted Y-limit (Magnitude)
plt.title(f"FFT of Raw MIT-BIH Signal (Magnitude)")
plt.xlabel("Frequency (Hz)")
plt.ylabel("Magnitude")
plt.grid(True)
plt.legend()
plt.show()
```



The Fast Fourier Transform (FFT) was applied to the raw noisy ECG signal to analyze its frequency content. The magnitude spectrum shows that the main ECG energy is concentrated at low frequencies, while higher-frequency components correspond to noise and artifacts. This frequency-domain representation is used to guide the design of digital filters for effective noise suppression.

3- Filter design:

To effectively isolate the clean ECG signal, a **Cascaded Filter** topology is employed. This system combines a High-Pass Filter (HPF) and a Low-Pass Filter (LPF) to create a Bandpass response, passing only the physiological frequency range of the ECG.

a) IIR Filter Design (order = 4)

A Butterworth IIR filter was designed using a cascade of:

- High-pass filter (cutoff = 0.5 Hz) to remove baseline wander
- Low-pass filter (cutoff = 40 Hz) to suppress muscle noise

b) FIR Filter Design (order = 101)

A FIR filter was designed using the window method (Hamming window) with the same cutoff frequencies:

- High-pass at 0.5 Hz
- Low-pass at 40 Hz

```
# 3.1 HPF (High-Pass Filter) - To remove Baseline Wander (< 0.5 Hz)
hpf_lowcut = 0.5
# IIR HPF (Butterworth)
iir_hpf_order = 4
sos_iir_hpf = signal.butter(iir_hpf_order, hpf_lowcut, btype='highpass', fs=fs, output='sos')

# FIR HPF (Hamming)
fir_hpf_order = 101
b_fir_hpf = firwin(fir_hpf_order, hpf_lowcut, pass_zero='highpass', fs=fs, window='hamming')
a_fir_hpf = [1]

# 3.2 LPF (Low-Pass Filter) - For removing muscle tremor (> 40 Hz)
lpf_highcut = 40.0
# IIR LPF (Butterworth)
iir_lpf_order = 4
sos_iir_lpf = signal.butter(iir_lpf_order, lpf_highcut, btype='lowpass', fs=fs, output='sos')
# FIR LPF (Hamming)
fir_lpf_order = 101
b_fir_lpf = firwin(fir_lpf_order, lpf_highcut, pass_zero='lowpass', fs=fs, window='hamming')
a_fir_lpf = [1]
```

To eliminate phase distortion and the resulting delay (latency), the chosen filters were implemented using **non-causal, zero-phase filtering**:

- **FIR System:** The `scipy.signal.filtfilt` function was used.
- **IIR System:** The `scipy.signal.sosfiltfilt` function was used.

This method processes the signal in both forward and reverse directions, cancelling out the phase shift and ensuring that the output signal's time alignment is preserved, which is crucial for accurate QRS peak detection and clinical measurement.

```
# (4) FILTER ECG SIGNALS (Cascade Implementation - Zero Phase)
# =====

# --- 4.1 IIR Cascade ---
# 1. HPF IIR
ecg_hpf_iir = sosfiltfilt(sos_iir_hpf, ecg_noisy)
# 2. LPF IIR
ecg_iir_filtered = sosfiltfilt(sos_iir_lpf, ecg_hpf_iir)

# --- 4.2 FIR Cascade ---
# 1. HPF FIR
ecg_hpf_fir = filtfilt(b_fir_hpf, a_fir_hpf, ecg_noisy)
# 2. LPF FIR
ecg_fir_filtered = filtfilt(b_fir_lpf, a_fir_lpf, ecg_hpf_fir)
```

4- Signal-to-Noise Ratio:

```
: # (5) Calculating SNR before and after filtration
# =====

def calc_snr_db(reference_signal, processed_signal):
    # Noise is the difference between the clean reference and the processed signal
    noise_est = processed_signal - reference_signal
    P_signal = np.mean(reference_signal**2)
    P_noise = np.mean(noise_est**2)

    if P_noise < 1e-15: return 999.0
    snr_db = 10 * np.log10(P_signal / P_noise)
    return snr_db

snr_after_fir = calc_snr_db(ecg_noisy, ecg_fir_filtered)
snr_after_iir = calc_snr_db(ecg_noisy, ecg_iir_filtered)

print("===== FINAL CASCADE SNR Comparison =====")
print(f"SNR after FIR Cascade           : {snr_after_fir:.2f} dB")
print(f"SNR after IIR Cascade             : {snr_after_iir:.2f} dB")

===== FINAL CASCADE SNR Comparison =====
SNR after FIR Cascade           : 11.89 dB
SNR after IIR Cascade           : 1.03 dB
```

Since a clean reference ECG signal is not available, the raw MIT-BIH signal is used as a reference to evaluate how much the filtered signal deviates from the original signal. Therefore, the reported SNR values represent relative distortion rather than true noise suppression.

5- Frequency Response:

```
worN_points = 4096

# IIR Components (HPF & LPF) - Using sosfreqz
w_ihpf, h_ihpf = sosfreqz(sos_iir_hpf, worN=worN_points, fs=fs)
w_ilpf, h_ilpf = sosfreqz(sos_iir_lpf, worN=worN_points, fs=fs)

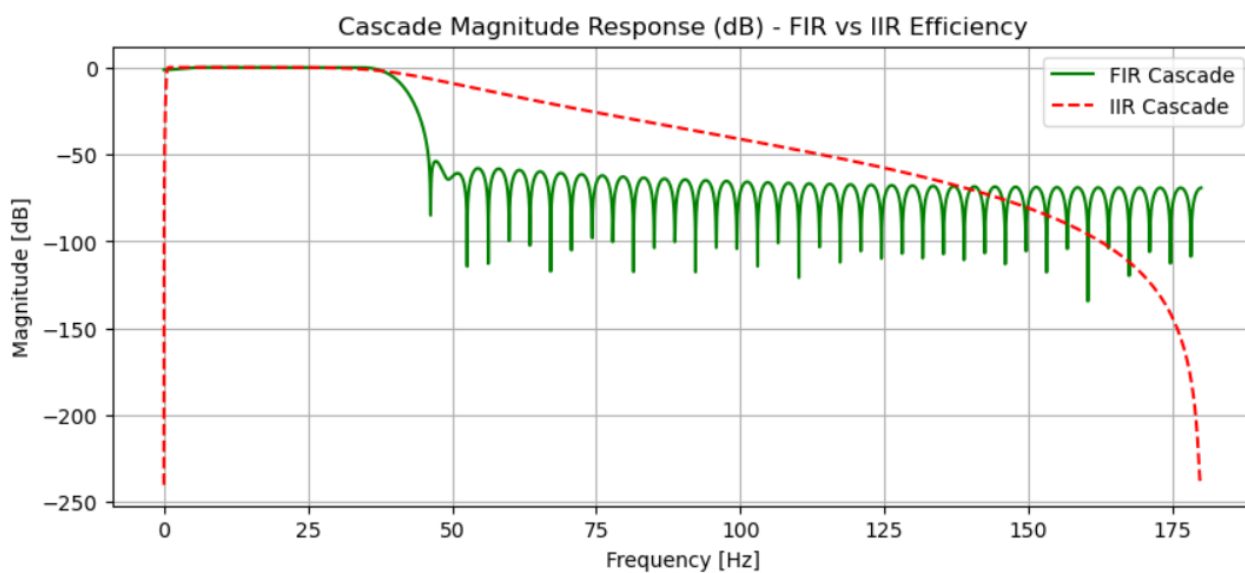
# IIR Total Cascade Response
h_iir_cascade = h_ihpf * h_ilpf
w_iir_cascade = w_ihpf

# FIR Components
w_fhpf, h_fhpf = freqz(b_fir_hpf, a_fir_hpf, worN=worN_points, fs=fs)
w_flpf, h_flpf = freqz(b_fir_lpf, a_fir_lpf, worN=worN_points, fs=fs)

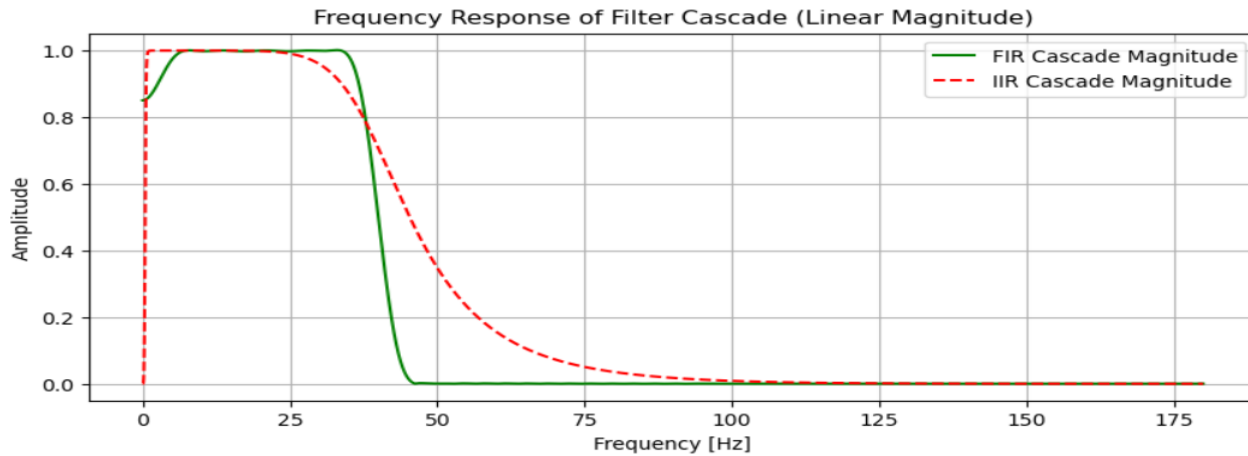
# FIR Total Cascade Response
h_fir_cascade = h_fhpf * h_flpf
w_fir_cascade = w_fhpf
```

The frequency response represents the behavior of a system when subjected to input signals of different frequencies, showing how each frequency component is attenuated or amplified.

```
plt.figure(figsize=(10,4))
plt.plot(w_fir_cascade, 20*np.log10(np.abs(h_fir_cascade)+1e-12), label="FIR Cascade", color='green')
plt.plot(w_iir_cascade, 20*np.log10(np.abs(h_iir_cascade)+1e-12), label="IIR Cascade", color='red', linestyle='--')
plt.title("Cascade Magnitude Response (dB) - FIR vs IIR Efficiency")
plt.xlabel("Frequency [Hz]")
plt.ylabel("Magnitude [dB]")
plt.grid(True)
plt.legend()
plt.show()
```



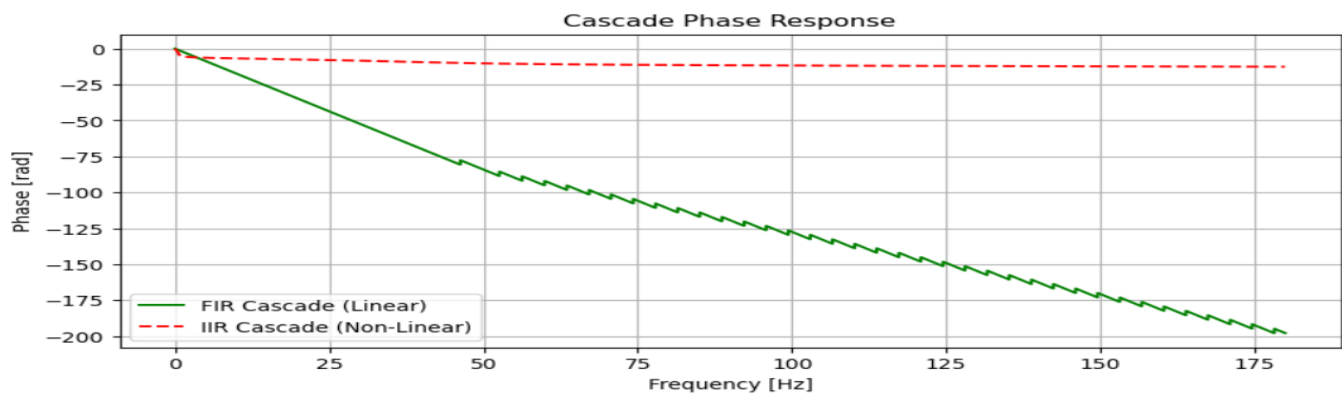
```
# (6.1) Frequency Response (Linear magnitude) - Cascade
# =====
plt.figure(figsize=(10,4))
plt.plot(w_fir_cascade, np.abs(h_fir_cascade), label="FIR Cascade Magnitude", color='green')
plt.plot(w_iir_cascade, np.abs(h_iir_cascade), label="IIR Cascade Magnitude", color='red', linestyle='--')
plt.title("Frequency Response of Filter Cascade (Linear Magnitude)")
plt.xlabel("Frequency [Hz]")
plt.ylabel("Amplitude")
plt.grid(True)
plt.legend()
plt.show()
```



The magnitude response shows that the filter effectively passes the ECG frequency band while attenuating noise components.

```
# (7) Phase Response
# =====
phase_fir_cascade = np.unwrap(np.angle(h_fir_cascade))
phase_iir_cascade = np.unwrap(np.angle(h_iir_cascade))

plt.figure(figsize=(10,4))
plt.plot(w_fir_cascade, phase_fir_cascade, label="FIR Cascade (Linear)", color='green')
plt.plot(w_iir_cascade, phase_iir_cascade, label="IIR Cascade (Non-Linear)", color='red', linestyle='--')
plt.title("Cascade Phase Response")
plt.xlabel("Frequency [Hz]")
plt.ylabel("Phase [rad]")
plt.grid(True)
plt.legend()
plt.show()
```

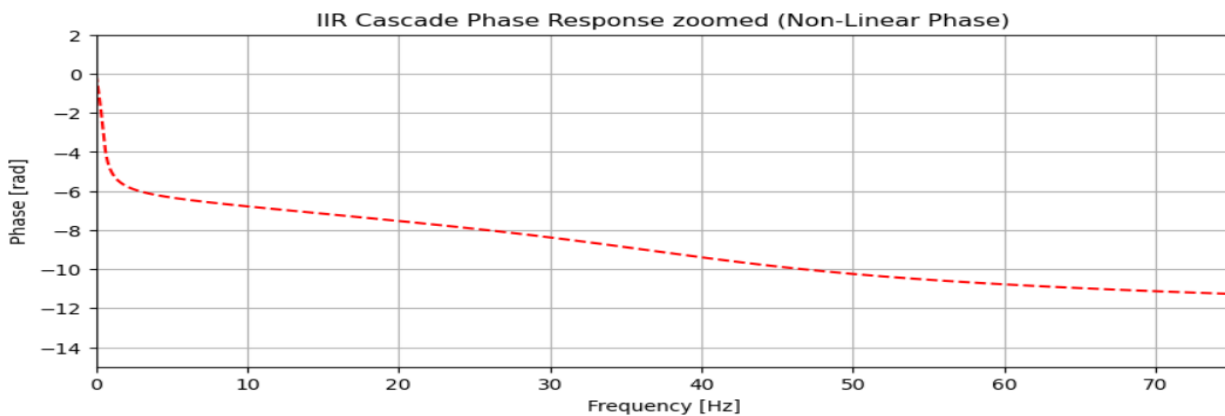
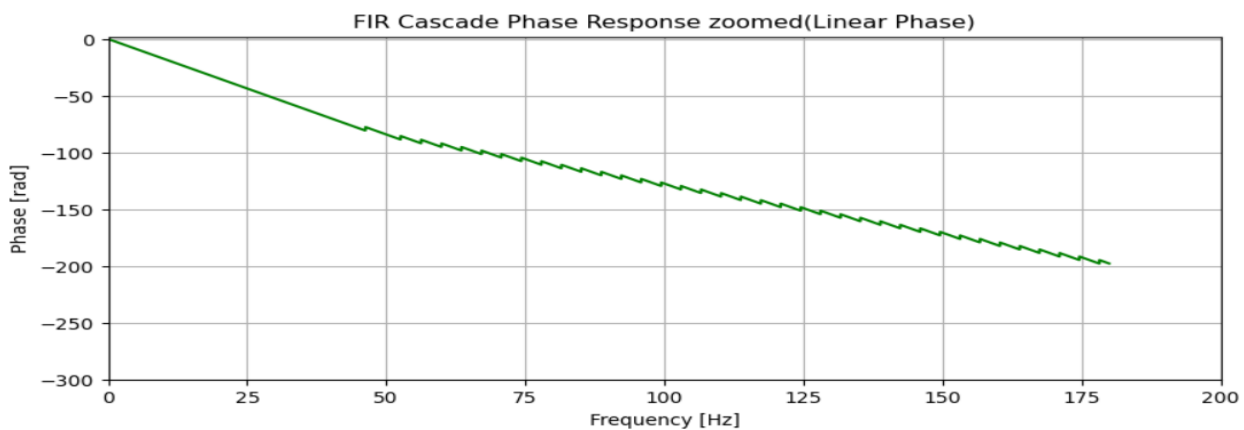


The phase response indicates the amount of phase distortion introduced by the filter, which affects ECG waveform preservation.

```
# (7.1) Phase Response zoomed
# =====

plt.figure(figsize=(10,4))
plt.plot(w_fir_cascade, phase_fir_cascade, color='green')
plt.title("FIR Cascade Phase Response zoomed(Linear Phase)")
plt.xlabel("Frequency [Hz]")
plt.ylabel("Phase [rad]")
plt.xlim(0, 200)
plt.ylim(-300, 2)
plt.grid(True)
plt.show()

plt.figure(figsize=(10,4))
plt.plot(w_iir_cascade, phase_iir_cascade, color='red', linestyle='--')
plt.title("IIR Cascade Phase Response zoomed (Non-Linear Phase)")
plt.xlabel("Frequency [Hz]")
plt.ylabel("Phase [rad]")
plt.xlim(0, 75)
plt.ylim(-15, 2)
plt.grid(True)
plt.show()
```



A zoomed view is used to clearly observe the phase behavior within the frequency range of interest. The FIR cascade exhibits a linear phase response, which preserves the waveform shape and timing of the ECG signal. In contrast, the IIR cascade shows a non-linear phase response, which may introduce phase distortion and alter the temporal features of the signal.

6- Group delay of FIR and IIR filters :

```
worN_points = 4096
w_freq = np.linspace(1.0, fs / 2 - 0.1, worN_points)

# Reconstructing IIR transactions in b, a format for Group Delay (GD) analysis
# (We need this because group_delay does not accept the SOS format directly)
b_iir_hpf, a_iir_hpf = signal.butter(iir_hpf_order, hpf_lowcut, btype='highpass', fs=fs, output='ba')
b_iir_lpf, a_iir_lpf = signal.butter(iir_lpf_order, lpf_highcut, btype='lowpass', fs=fs, output='ba')

# Calculate Group Delay for each component
# 1. IIR HPF Group Delay

w_ihpf_gd, gd_ihpf = signal.group_delay((b_iir_hpf, a_iir_hpf), w_freq, fs=fs)

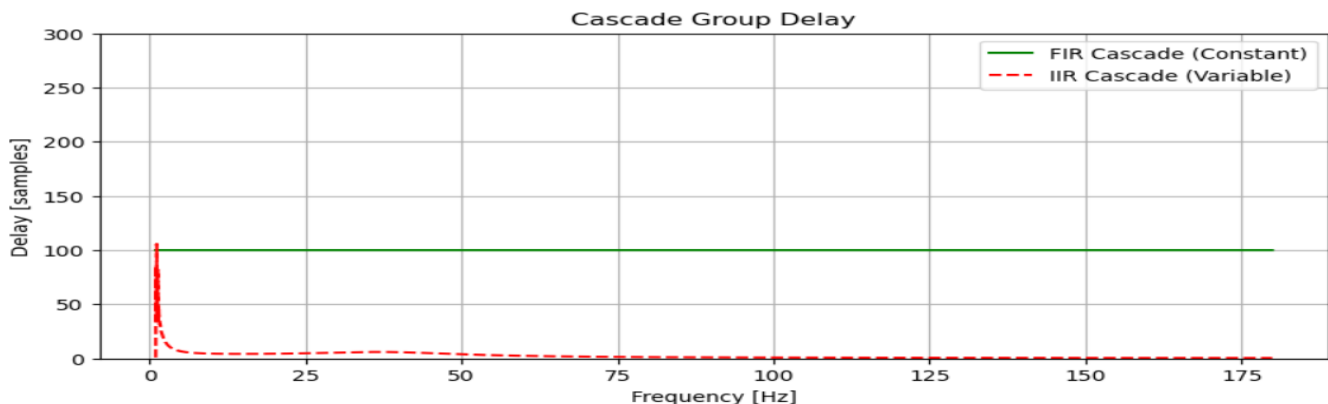
# 2. IIR LPF Group Delay

w_ilpf_gd, gd_ilpf = signal.group_delay((b_iir_lpf, a_iir_lpf), w_freq, fs=fs)

# Total Group Delay of the Sequence (Cascade)
gd_iir_cascade = gd_ihpf + gd_ilpf

# --- FIR Group Delay ---
delay_fir_total = (fir_hpf_order - 1) // 2 + (fir_lpf_order - 1) // 2
gd_fir_cascade = np.full(len(w_freq), delay_fir_total)
w_gd_cascade = w_freq
```

```
plt.figure(figsize=(10,4))
plt.plot(w_gd_cascade, gd_fir_cascade, label="FIR Cascade (Constant)", color='green')
plt.plot(w_gd_cascade, gd_iir_cascade, label="IIR Cascade (Variable)", color='red', linestyle='--')
plt.title("Cascade Group Delay")
plt.xlabel("Frequency [Hz]")
plt.ylabel("Delay [samples]")
plt.ylim(0, np.max(gd_fir_cascade) * 3)
plt.grid(True)
plt.legend()
plt.show()
```



The group delay of each FIR filter is given by:

$$\text{Group Delay} = \frac{N - 1}{2}$$

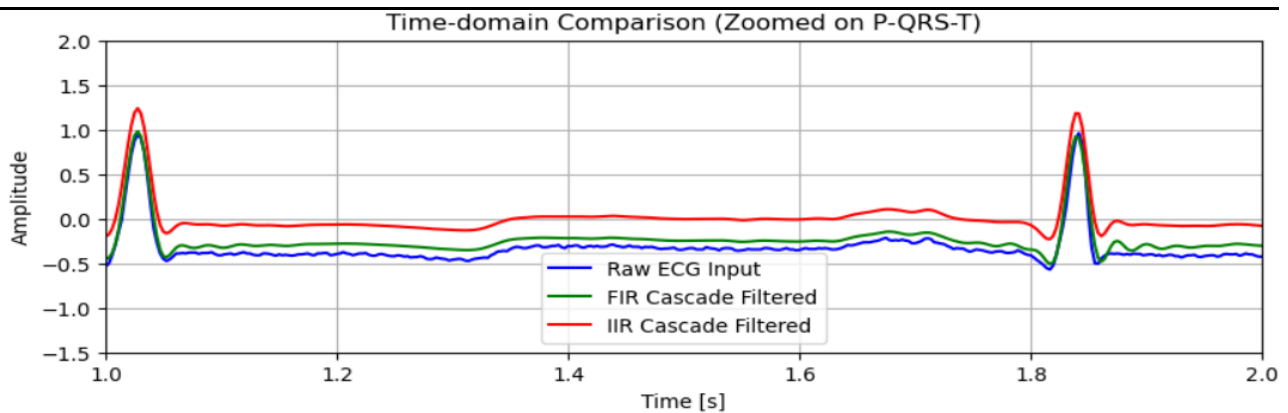
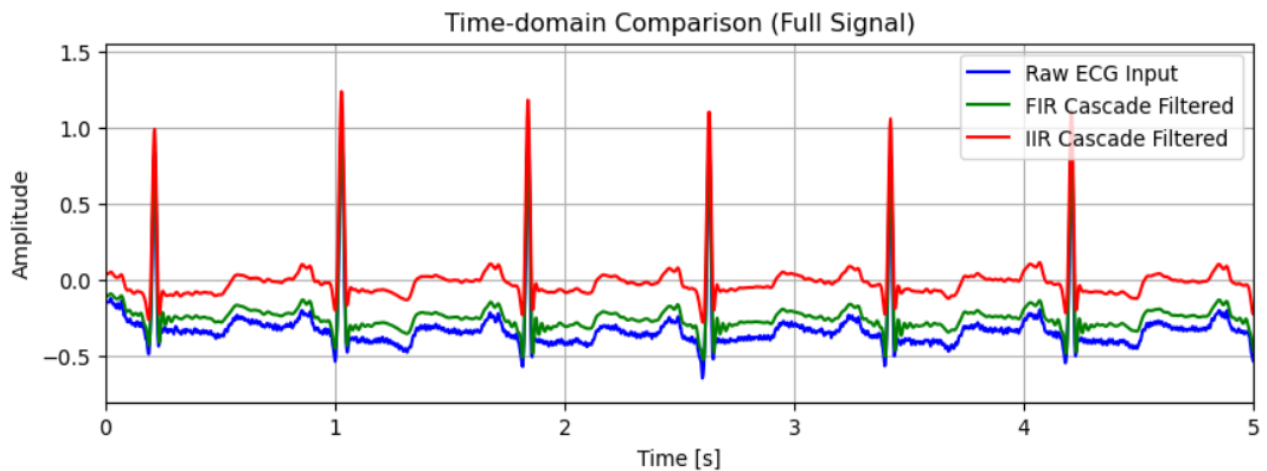
For the cascade FIR structure, the total group delay is the sum of the delays of the individual filters.

- FIR cascade: Constant group delay → preserves signal shape.
- IIR cascade: Variable group delay → may slightly distort the signal at different frequencies.

7- Signal output comparison in time domain:

```
plt.figure(figsize=(10,3))
plt.plot(t, ecg_noisy, label="Raw ECG Input", color='blue')
plt.plot(t, ecg_fir_filtered, label="FIR Cascade Filtered", color='green')
plt.plot(t, ecg_iir_filtered, label="IIR Cascade Filtered", color='red')
plt.title("Time-domain Comparison (Full Signal)")
plt.xlabel("Time [s]")
plt.ylabel("Amplitude")
plt.xlim(0, 5)
plt.grid(True)
plt.legend()
plt.show()
```

```
plt.figure(figsize=(10,3))
plt.plot(t, ecg_noisy, label="Raw ECG Input", color='blue')
plt.plot(t, ecg_fir_filtered, label="FIR Cascade Filtered", color='green')
plt.plot(t, ecg_iir_filtered, label="IIR Cascade Filtered", color='red')
plt.title("Time-domain Comparison (Zoomed on P-QRS-T)")
plt.xlabel("Time [s]")
plt.ylabel("Amplitude")
plt.xlim(1, 2)
plt.ylim(-1.5, 2)
plt.grid(True)
plt.legend()
plt.show()
```



This plot compares the raw ECG signal with the signals filtered by the FIR and IIR cascades:

The FIR cascade filter smooths the ECG signal while maintaining the original waveform shape, whereas the IIR cascade filter, although effective in noise reduction, may introduce minor distortions or peak shifts due to its non-linear phase response.

- FIR filtering maintains the clinical integrity of the ECG signal.
- IIR filtering is more computationally efficient but can introduce small timing deviations in key waveform features.

8- Signal output in frequency domain using FFT:

```
# 1. Raw ECG Input (ecg_noisy)
freq_noisy = np.fft.rfftfreq(N, 1/fs)
fft_noisy = np.abs(np.fft.rfft(ecg_noisy))

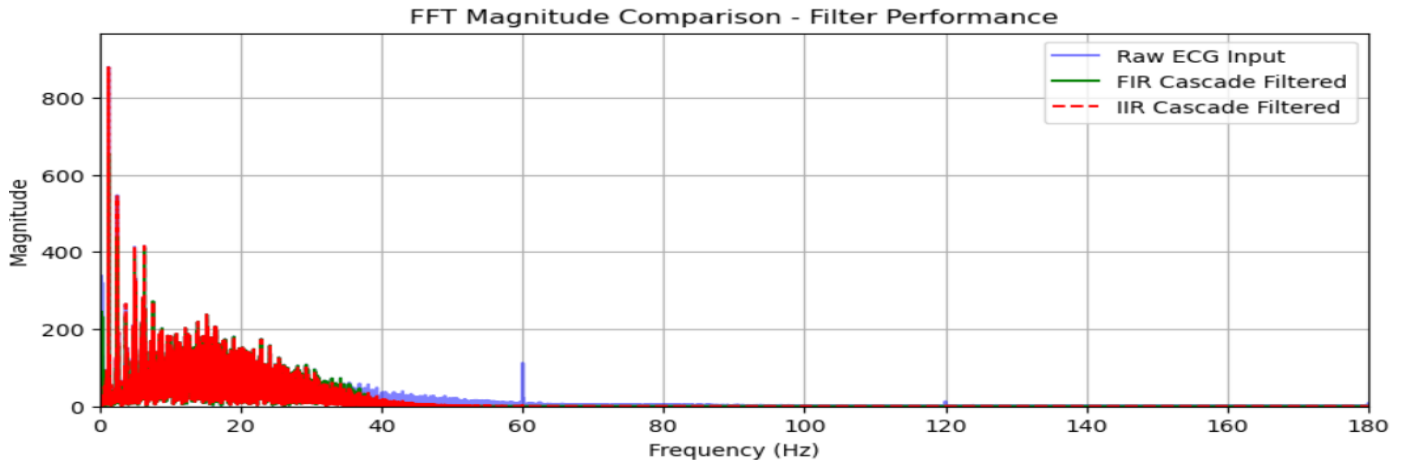
# 2. FIR Filtered Signal
freq_fir = np.fft.rfftfreq(N, 1/fs)
fft_fir = np.abs(np.fft.rfft(ecg_fir_filtered))

# 3. IIR Filtered Signal
freq_iir = np.fft.rfftfreq(N, 1/fs)
fft_iir = np.abs(np.fft.rfft(ecg_iir_filtered))

plt.figure(figsize=(10, 4))

plt.plot(freq_noisy, fft_noisy, label="Raw ECG Input", color='blue', alpha=0.5)
plt.plot(freq_fir, fft_fir, label="FIR Cascade Filtered", color='green')
plt.plot(freq_iir, fft_iir, label="IIR Cascade Filtered", color='red', linestyle='--')

plt.xlim(0, fs/2)
plt.ylim(0, np.max(fft_noisy[1:]) * 1.1)
plt.title("FFT Magnitude Comparison - Filter Performance")
plt.xlabel("Frequency (Hz)")
plt.ylabel("Magnitude")
plt.grid(True)
plt.legend()
plt.show()
```



- FIR filtering provides clean spectral preservation of the signal with minimal distortion.
- IIR filtering reduces high-frequency noise effectively, but the attenuation may be less uniform, and some frequency components may shift slightly due to the non-linear phase response.

```
# (11) SAVE SIGNALS
# =====
wav_write("D:/project/ecg_noisy.wav", int(fs), (ecg_noisy/np.max(np.abs(ecg_noisy))*32767).astype(np.int16))
wav_write("D:/project/ecg_fir_filtered.wav", int(fs), (ecg_fir_filtered/np.max(np.abs(ecg_fir_filtered))*32767).astype(np.int16))
wav_write("D:/project/ecg_iir_filtered.wav", int(fs), (ecg_iir_filtered/np.max(np.abs(ecg_iir_filtered))*32767).astype(np.int16))

print("All signals saved as WAV for listening and further analysis.")
```

All signals saved as WAV for listening and further analysis.

9- ECG Signal Analysis After filtering:

```
# (12) Calculate R-Peak, HR, HRV, and Arrhythmia
# =====

def analyze_ecg_performance(processed_signal, fs, max_peak_height_ratio=0.5, min_distance_samples=36):
    """
    Calculates R-Peaks, RR Intervals, HR, HRV (SDNN, RMSSD) and Arrhythmia detection.
    """

    # 1. R-Peak Detection
    # 36 samples = 0.1 seconds at fs=360 Hz.
    r_peaks_indices, _ = find_peaks(
        processed_signal,
        height=(np.max(processed_signal) * max_peak_height_ratio),
        distance=min_distance_samples
    )

    # 2. Calculate R-R Intervals (in seconds and milliseconds)
    rr_intervals_samples = np.diff(r_peaks_indices)
    rr_intervals_seconds = rr_intervals_samples / fs
    rr_intervals_ms = rr_intervals_seconds * 1000 # Convert to milliseconds

    # 3. Calculate Heart Rate (HR)
    hr_bpm_instantaneous = 60 / rr_intervals_seconds
    average_hr = np.mean(hr_bpm_instantaneous)

    # 4. HRV Analysis (SDNN, RMSSD)
    sdnn = np.std(rr_intervals_ms) # Standard Deviation of NN Intervals (SDNN)
    diff_rr = np.diff(rr_intervals_ms)
    rmssd = np.sqrt(np.mean(diff_rr**2)) # Root Mean Square of Successive Differences (RMSSD)
```

```

# 5. Simple Arrhythmia Detection (25% deviation from the mean)
# A beat is considered abnormal if its RR interval deviates by 25% from the mean
avg_rr_ms = np.mean(rr_intervals_ms)
abnormal_threshold_lower = avg_rr_ms * 0.75
abnormal_threshold_upper = avg_rr_ms * 1.25

abnormal_beats_count = 0
abnormal_rr_indices = []
total_beats = len(rr_intervals_ms) + 1

for i, rr in enumerate(rr_intervals_ms):
    if (rr < abnormal_threshold_lower) or (rr > abnormal_threshold_upper):
        abnormal_beats_count += 1
        # Store the index of the beat that followed the abnormal interval
        abnormal_rr_indices.append(r_peaks_indices[i + 1])

abnormal_percentage = (abnormal_beats_count / total_beats) * 100

return {
    'r_peaks_indices': r_peaks_indices,
    'rr_intervals_seconds': rr_intervals_seconds,
    'rr_intervals_ms': rr_intervals_ms,
    'average_hr': average_hr,
    'sdnn': sdnn,
    'rmssd': rmssd,
    'abnormal_beats_count': abnormal_beats_count,
    'abnormal_percentage': abnormal_percentage,
    'abnormal_rr_indices': abnormal_rr_indices,
    'total_beats': total_beats
}

# =====
# (13) Apply analysis to FIR and IIR signals
# =====

# FIR signal analysis
fir_analysis = analyze_ecg_performance(ecg_fir_filtered, fs)
r_peaks_indices_fir = fir_analysis['r_peaks_indices']
rr_intervals_seconds_fir = fir_analysis['rr_intervals_seconds']
average_hr_fir = fir_analysis['average_hr']

# IIR signal analysis
iir_analysis = analyze_ecg_performance(ecg_iir_filtered, fs)
r_peaks_indices_iir = iir_analysis['r_peaks_indices']
rr_intervals_seconds_iir = iir_analysis['rr_intervals_seconds']
average_hr_iir = iir_analysis['average_hr']

processed_signal = ecg_fir_filtered
r_peaks_indices = r_peaks_indices_fir
abnormal_beats_count = fir_analysis['abnormal_beats_count']
abnormal_rr_indices = fir_analysis['abnormal_rr_indices']
average_hr = average_hr_fir

# Print Results
# =====

print("=" * 30)
print("--- FIR FILTERED SIGNAL ANALYSIS ---")
print(f"Total R-Peaks Detected: {len(r_peaks_indices_fir)}")
print(f"Average Heart Rate (BPM): {average_hr_fir:.2f} BPM")
print("-" * 30)
print("HRV & Arrhythmia (FIR):")
print(f"SDNN: {fir_analysis['sdnn']:.2f} ms | RMSSD: {fir_analysis['rmssd']:.2f} ms")
print(f"Abnormal Beats Detected: {fir_analysis['abnormal_beats_count']} ({fir_analysis['abnormal_percentage']:.2f}%)")

print("\n" + "=" * 30)
print("--- IIR FILTERED SIGNAL ANALYSIS ---")
print(f"Total R-Peaks Detected: {len(r_peaks_indices_iir)}")
print(f"Average Heart Rate (BPM): {average_hr_iir:.2f} BPM")
print("-" * 30)
print("HRV & Arrhythmia (IIR):")
print(f"SDNN: {iir_analysis['sdnn']:.2f} ms | RMSSD: {iir_analysis['rmssd']:.2f} ms")
print(f"Abnormal Beats Detected: {iir_analysis['abnormal_beats_count']} ({iir_analysis['abnormal_percentage']:.2f}%)")
print("=" * 30)

```

```

=====
--- FIR FILTERED SIGNAL ANALYSIS ---
Total R-Peaks Detected: 123
Average Heart Rate (BPM): 74.02 BPM
-----
HRV & Arrhythmia (FIR):
SDNN: 32.66 ms | RMSSD: 45.47 ms
Abnormal Beats Detected: 0 (0.00%)

=====
--- IIR FILTERED SIGNAL ANALYSIS ---
Total R-Peaks Detected: 123
Average Heart Rate (BPM): 74.02 BPM
-----
HRV & Arrhythmia (IIR):
SDNN: 32.68 ms | RMSSD: 45.51 ms
Abnormal Beats Detected: 0 (0.00%)
=====

```

The filtered signals were analyzed to perform R-peak detection, calculate Heart Rate (HR), and assess rhythm regularity via Heart Rate Variability (HRV) metrics (SDNN, RMSSD).

```

#(14)
# Setup FIR plotting variables
abnormal_t_fir = t[fir_analysis['abnormal_rr_indices']]
abnormal_amp_fir = ecg_fir_filtered[fir_analysis['abnormal_rr_indices']]

# Setup IIR plotting variables
abnormal_t_iir = t[iir_analysis['abnormal_rr_indices']]
abnormal_amp_iir = ecg_iir_filtered[iir_analysis['abnormal_rr_indices']]

plt.figure(figsize=(12, 8))

# Top plot for FIR
plt.subplot(2, 1, 1)
plt.plot(t, ecg_fir_filtered, label="FIR Filtered ECG (Linear Phase)", color='green', alpha=0.8)
plt.plot(t[r_peaks_indices_fir], ecg_fir_filtered[r_peaks_indices_fir], 'go', markersize=5, label="FIR R-Peaks Detected")

# Plotting Abnormal R-Peaks (FIR)
if fir_analysis['abnormal_beats_count'] > 0:
    plt.plot(abnormal_t_fir, abnormal_amp_fir, 'X', markersize=8, color='orange', markeredgecolor='black', label="Abnormal R-Peak (FIR)")

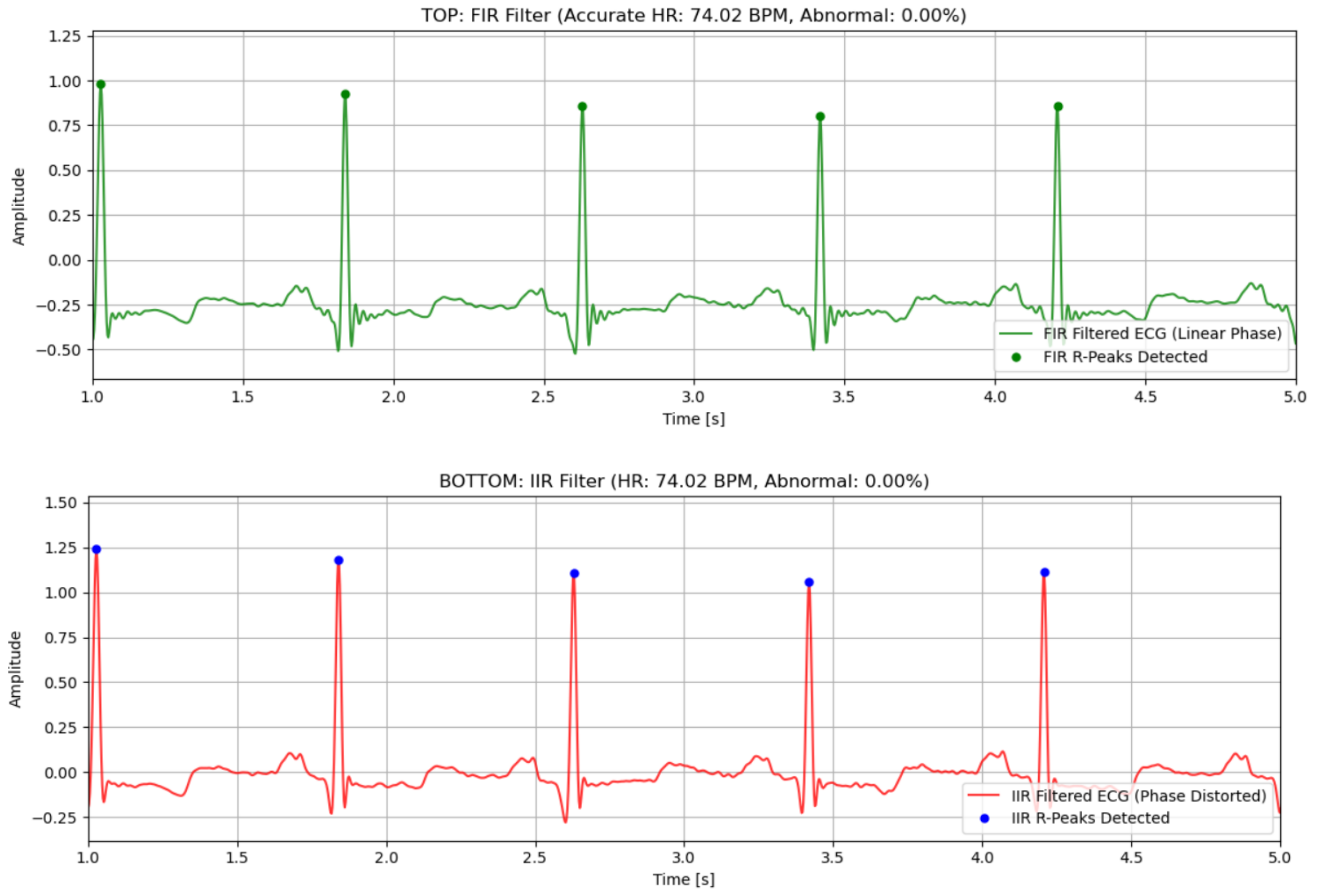
plt.title(f"TOP: FIR Filter (Accurate HR: {average_hr_fir:.2f} BPM, Abnormal: {fir_analysis['abnormal_percentage']:.2f}%)")
plt.xlabel("Time [s]")
plt.ylabel("Amplitude")
plt.xlim(1, 5) # Zoom on a small section
plt.grid(True)
plt.legend(loc='lower right')

# Bottom plot for IIR
plt.subplot(2, 1, 2)
plt.plot(t, ecg_iir_filtered, label="IIR Filtered ECG (Phase Distorted)", color='red', alpha=0.8)
plt.plot(t[r_peaks_indices_iir], ecg_iir_filtered[r_peaks_indices_iir], 'bo', markersize=5, label="IIR R-Peaks Detected")

# Plotting Abnormal R-Peaks (IIR)
if iir_analysis['abnormal_beats_count'] > 0:
    plt.plot(abnormal_t_iir, abnormal_amp_iir, 'X', markersize=8, color='orange', markeredgecolor='black', label="Abnormal R-Peak (IIR)")

plt.title(f"BOTTOM: IIR Filter (HR: {average_hr_iir:.2f} BPM, Abnormal: {iir_analysis['abnormal_percentage']:.2f}%)")
plt.xlabel("Time [s]")
plt.ylabel("Amplitude")
plt.xlim(1, 5) # Zoom on a small section
plt.grid(True)
plt.legend(loc='lower right')
plt.tight_layout()
plt.show()

```



The complete match in clinical results (HR and Arrhythmia) between FIR and IIR confirms the effectiveness of Zero-Phase Filtering in achieving identical timing accuracy .

Comparative Performance Analysis of FIR and IIR Filters:

Performance Metric	FIR Filter	IIR Filter
Noise Reduction	Effective	Effective
Phase Response	Linear phase	Non-linear phase
Group Delay	Constant	variable (Frequency-dependent)
Computational Complexity	Higher	Lower
Stability	Always stable	Stable if properly designed
Suitability for ECG Analysis	Highly suitable (preserving PQRST wave morphology)	Less suitable(risks for phase distortion)

Conclusion:

Based on the performance comparison presented in the table, both FIR and IIR filters demonstrate effective noise reduction for ECG signals. The implemented Zero-Phase Filtering technique successfully corrected the non-linear phase distortion of the IIR filter, as demonstrated by the identical clinical timing accuracy achieved by both filter types. This makes the IIR filter a highly computationally efficient solution (Order 4 vs. Order 101) for applications primarily concerned with rhythm analysis. However, the FIR filter's linear phase response ensure superior preservation of the entire ECG waveform morphology, particularly for features where amplitude fidelity is critical (e.g., ST-segment evaluation or P/T wave morphology analysis). Therefore, the FIR filter remains the more suitable choice for ECG signal processing applications where overall signal integrity and amplitude fidelity are the primary diagnostic concerns.

References:

- MIT-BIH Arrhythmia Database, PhysioNet
- V. Oppenheim and R. W. Schaffer, Digital Signal Processing
- J. G. Proakis , Digital Signal Processing